



向线程池提交任务（下）

北京理工大学计算机学院
金旭亮

复习: Callable与Future的关系



我们可以用`Future.get()`来获取`Callable`接口返回的执行结果, 通过`Future.isDone()`来判断任务是否已经执行完了。



在`call()`未执行完毕之前, 调用`get()`的线程会被阻塞, 直到`call()`方法返回了结果之后, `Future.get()`才会得到结果, 调用者线程才会回到`runnable`状态。



`Future`可以看成是一个结果存储器, 它存储了`call()`这个任务的结果, 而这个任务的执行时间是无法提前确定的, 因为这完全取决于`call()`方法执行的情况。

Future.get()方法详解



get方法的行为特性取决于Callable任务的状态，有以下五种情况：

(1) 任务正常完成，get方法会立刻返回结果

(2) 任务尚未完成（任务还没开始或进行中），get方法将阻塞并直到任务完成。

(3) 任务执行过程中抛出Exception：这时，get方法会发现这点，然后抛出ExecutionException。

(4) 任务被取消，get方法会抛出CancellationException。

(5) 任务超时，get方法有一个重载形式，可以指定等待结果的时间，超时会抛出TimeoutException。

返回多个任务的执行结果

//一个返回随机数的计算任务

```
1 usage
static class CallableTask implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        Thread.sleep( millis: 3000);
        return new Random().nextInt();
    }
}
```

在实际开发中，要注意这种方式的弊端：

如果排在前面的get()方法长期阻塞，会导致后面的计算结果被迫“等待”

```
public static void main(String[] args) throws InterruptedException {
    //可以试着减少线程池中的线程数目，会发现结果是分批呈现的
    ExecutorService service = Executors.newFixedThreadPool( nThreads: 20);
    ArrayList<Future<Integer>> futures = new ArrayList<>();
    //由于线程池中有20个任务，所以，这20个任务是并行执行的
    for (int i = 0; i < 20; i++) {
        Future<Integer> future = service.submit(new CallableTask());
        futures.add(future);
    }
    Thread.sleep( millis: 5000);
    //休眠5秒后，由于所有并行任务都已经执行完毕，所以，结果是立即出现的。
    for (int i = 0; i < 20; i++) {
        //依次提取出所有任务的结果
        Future<Integer> future = futures.get(i);
        try {
            Integer integer = future.get();
            System.out.println(integer);
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }
    }
}
```

MultiFutures.java

如果任务中抛出了异常.....

```
1 usage
static class CallableTask implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        throw new IllegalArgumentException("Callable抛出异常");
    }
}
```



虽然任务很早就抛出了illegal...异常，但要等到调用get方法时才会抛出，并且抛出的异常是ExecutionException。

```
public static void main(String[] args) {
    ExecutorService service = Executors.newFixedThreadPool( nThreads: 20);
    Future<Integer> future = service.submit(new CallableTask());
    try {
        for (int i = 0; i < 5; i++) {
            System.out.println(i);
            Thread.sleep( millis: 500);
        }
        //不管有没有异常抛出，只要线程结束了工作，isDone属性都是true
        System.out.println(future.isDone());
        //虽然在线程池执行计算任务时异常已经抛出，但一定要等到get时才能捕获
        future.get();
    } catch (InterruptedException e) {
        e.printStackTrace();
        System.out.println("InterruptedException异常");
    } catch (ExecutionException e) {
        e.printStackTrace();
        System.out.println("ExecutionException异常");
    }
}
```

ThrowExceptionDemo.java

处理超时-1

一个运行时间较长的数据处理任务:

```
static class FetchInfoTask implements Callable<Info> {  
    @Override  
    public Info call() throws Exception {  
        try {  
            System.out.println("等待3秒出结果...");  
            Thread.sleep( millis: 3000);  
        } catch (InterruptedException e) {  
            System.out.println("sleep期间被中断了");  
        }  
        return new Info("计算任务顺利执行结束");  
    }  
}
```

TestTimeout.java

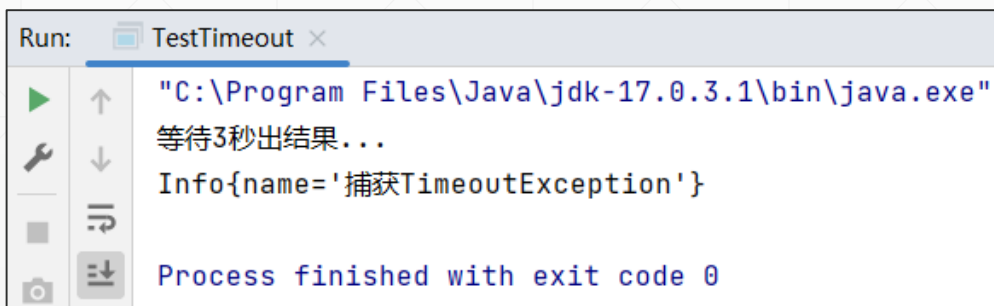
Info类封装一个字符串，用作任务提示信息：

```
19 usages  
static class Info {  
    2 usages  
    String info;  
    9 usages  
    public Info(String info) {  
        this.info = info;  
    }  
    @Override  
    public String toString() {  
        return "Info{" +  
            "name='" + info + '\'' +  
            '}';  
    }  
}
```

处理超时-2

```
public static void main(String[] args)
    throws InterruptedException {
    TestTimeout timeout = new TestTimeout();
    timeout.printInfo();
}
```

运行截图：



2 usages

```
public void printInfo() throws InterruptedException {
    Future<Info> f = threadPool.submit(new FetchInfoTask());
    Info info;
    try {
        //最长等待2秒
        info = f.get( timeout: 2000, TimeUnit.MILLISECONDS);
    } catch (InterruptedException e) {
        info = new Info("捕获InterruptedException");
    } catch (ExecutionException e) {
        info = new Info("捕获ExecutionException");
    } catch (TimeoutException e) {
        info = new Info("捕获TimeoutException");
    }
    //关闭线程池
    threadPool.shutdown();
    //等待线程池的中止
    threadPool.awaitTermination( timeout: 10, TimeUnit.SECONDS);
    //输出结果
    System.out.println(info);
}
```

提前取消-1



```
1 usage
static class FetchInfoTask implements Callable<Info> {
    @Override
    public Info call() throws Exception {
        //Thread.sleep是支持被中断的,
        //如果是其它代码, 则可以调用Thread.interrupted()
        //看看是否外界发出了中断请求
        try {
            System.out.println("等待3秒出结果...");
            Thread.sleep( millis: 3000);
            //当被取消之后, 这句是不会运行的
            System.out.println("在Thread.sleep之后运行");
        } catch (InterruptedException e) {
            System.out.println("sleep期间被中断了");
        }
        return new Info("计算任务顺利执行结束");
    }
}
```

如果希望任务可以被提前取消, 则应该在call方法中封装“任务取消”的处理逻辑, 其编程模式, 与“提前中断线程执行”完全一致。

TestCancel.java

提前取消-2

调用Future.cancel()
方法取消任务的执行。

cancel()方法的参数为
true还是false，对执
行结果有影响.....

```
public void cancelTask() throws InterruptedException {
    Future<Info> f = exec.submit(new FetchInfoTask());
    Info info = null;
    try {
        System.out.println("等待1秒后取消计算任务");
        Thread.sleep( millis: 1000);
        //当cancel方法参数为true时，线程会收到中断请求，否则，会自顾自地运行到结束
        boolean cancel = f.cancel( mayInterruptIfRunning: true);
        System.out.println("cancel的结果: " + cancel);
        //调用cancel方法之后，再次尝试调用get，将引发CancellationException
        info = f.get();
    } catch (InterruptedException e) {
        info=new Info("捕获InterruptedException");
    }
    catch (ExecutionException e) {
        info=new Info("捕获ExecutionException");
    }
    catch (CancellationException e) {
        info = new Info("捕获CancellationException");
    }
    exec.shutdown();
    //等待线程池的中止
    exec.awaitTermination( timeout: 10, TimeUnit.SECONDS);
    System.out.println(info);
}
```

运行截图



`f.cancel(true)`

```
Run: TestCancel x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
等待1秒后取消计算任务
等待3秒出结果...
sleep期间被中断了
cancel的结果: true
Info{name='捕获CancellationException'}
Process finished with exit code 0
```

`f.cancel(false)`

```
Run: TestCancel x
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
等待1秒后取消计算任务
等待3秒出结果...
cancel的结果: true
在Thread.sleep之后运行
Info{name='捕获CancellationException'}
Process finished with exit code 0
```

上面输出中的“在...之后运行”一句，是在`call()`方法中输出的，由于线程没有收到中断信号，所以输出了这句。

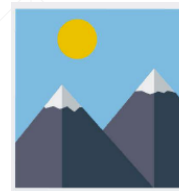
cancel()方法小结



cancel()方法主要用于通知线程池取消任务的执行：

1. 如果这个任务还没有执行，则任务会取消，方法返回true
2. 如果这个任务已经完成，或者已取消，那么cancel()方法返回false
3. 如果这个任务已经开始执行了，那么cancel()方法会尝试取消任务，其参数mayInterruptIfRunning的值，将决定是否向线程发出“中断”信号。

Cancel方法的应用场景

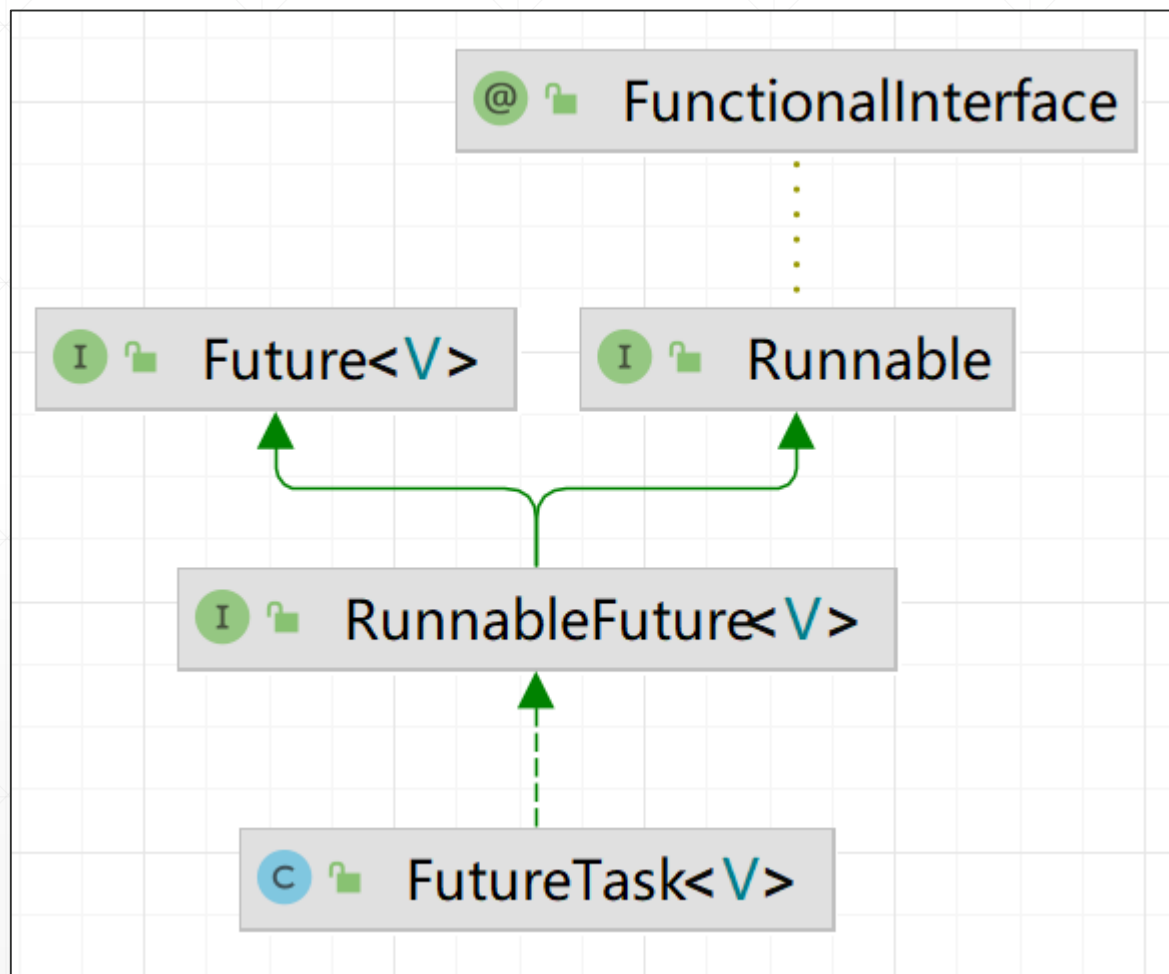


➔ `Future.cancel(true)` 适用于“任务能处理中断请求”的场景。

➔ `Future.cancel(false)` 用于避免启动尚未启动的任务，适用于：

1. 已经运行的任务没有代码处理中断
2. 需要等待已经开始的任务执行完成

身兼两职的FutureTask



JDK中提供了一个FutureTask类，它是一种包装器，同时实现了Runnable和Future接口.....

FutureTask可用于获取Future和任务的结果。

FutureTask示例

UseFutureTask.java

//一个简单的计算任务

2 usages

```
class Task implements Callable<Integer> {  
    @Override  
    public Integer call() throws Exception {  
        System.out.println("子线程正在计算");  
        Thread.sleep( millis: 3000);  
        int sum = 0;  
        for (int i = 1; i <= 100; i++) {  
            sum += i;  
        }  
        return sum;  
    }  
}
```

```
public class FutureTaskDemo {  
    public static void main(String[] args) {  
        Task task = new Task();  
        FutureTask<Integer> integerFutureTask = new FutureTask<>(task);  
        ExecutorService service = Executors.newCachedThreadPool();  
        service.submit(integerFutureTask);  
        try {  
            System.out.println("task运行结果: " + integerFutureTask.get());  
        } catch (InterruptedException | ExecutionException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

把Callable实例当作参数，生成FutureTask的对象，然后把这个对象当作一个Runnable对象，传给线程池去执行，最后通过FutureTask的get方法获取刚才执行的结果。

FutureTask的局限



不能清楚地表达多个FutureTask之间的关系



为了从FutureTask获取结果，我们必须调用get()方法，而该方法会在任务执行完毕前阻塞调用线程。

为了克服FutureTask的局限性，以及满足我们对异步编程的需要，JDK8中提供了CompletableFuture，它的用法比较复杂，放在单独的PPT中进行介绍。